ESP32 Edge Vision Station

10-Day NVIDIA Ignite Project Build Plan

Hardware constraint: Use only the components already in the Sundoofer kit. No plastics, blocks, 3D printing, additional sensors, or external physical construction.

Final Project Goal

Build a real-time embedded edge-vision station that senses an event, processes it locally, activates the camera, controls the servo, provides feedback through the display/buzzer, and uses an event-driven FreeRTOS architecture. If the exact ESP32/camera combination supports it, finish with basic local image processing or TinyML.

Core pipeline: Sense → Decide → Capture/Process → Act → Log

Day 1 — Hardware + Development Setup

• Identify the exact ESP32 variant, camera, PIR, IR sensor, servo, display/counter, buzzer, IR remote/receiver, and 74HC595.

• Install VS Code, ESP-IDF, ESP-IDF extension, Git, and required USB drivers.

• Create the GitHub repository: esp32-edge-vision-station.

• Create README.md, docs/, firmware/, hardware/, and tests/.

• Flash a basic program and confirm serial communication.

Deliverable: ESP32 successfully compiles, flashes, and prints a startup message.

Day 2 — GPIO + Basic Hardware Bring-Up

• Configure GPIO inputs and outputs.

• Test the PIR and IR obstacle sensor.

• Test the display/counter and buzzer.

• Print sensor states through the serial monitor.

• Learn digital logic and reliable input handling.

Deliverable: Working sensor-monitoring firmware with reliable GPIO behavior.

Day 3 — Interrupts + Event System

• Use a GPIO interrupt for the PIR.

• Create events such as MOTION_DETECTED, OBSTACLE_DETECTED, and REMOTE_COMMAND.

• Use a FreeRTOS queue to move events into an event manager.

• Add timestamped event logging.

Deliverable: Event-driven firmware that responds without blocking the processor.

Day 4 — Servo + PWM Control

• Configure PWM for the micro servo.

• Implement 0°, 45°, 90°, 135°, and 180° positions.

• Calibrate the usable range.

• Connect events to an automatic camera-position scan.

• Measure approximate servo response time.

Deliverable: ESP32 automatically positions/scans the servo in response to events.

Day 5 — IR Remote Control

• Decode the IR remote commands.

• Print protocol/address/command information.

• Map buttons to servo and system functions.

• Example: arrows control servo; OK triggers capture; 1=AUTO; 2=MANUAL; 3=SCAN.

• Implement operating-mode state management.

Deliverable: Full manual control through the IR remote.

Day 6 — Camera Bring-Up

• Initialize the camera independently.

• Capture a frame and verify its buffer and metadata.

• Record resolution and frame size.

• Measure capture latency.

• Monitor memory use and correctly release frame buffers.

Deliverable: Reliable camera acquisition with documented performance.

Day 7 — Full System Integration

• Combine PIR, IR sensor, camera, servo, display, buzzer, and remote.

• Implement: standby → motion → camera → servo scan → capture → standby.

• Use the display for STANDBY, MOTION, SCAN, and EVENT states.

• Use distinct buzzer notifications.

• Create an event counter.

Deliverable: First complete autonomous Edge Vision Station prototype.

Day 8 — FreeRTOS + Performance Engineering

• Separate Sensor, Camera, Remote, and Control/Decision functionality into tasks.

• Use FreeRTOS queues, timers, priorities, and non-blocking behavior.

• Measure sensor latency, event-queue latency, camera capture, servo response, RAM, and flash.

• Remove unnecessary delays/blocking code.

• Document the task architecture.

Deliverable: Proper real-time embedded architecture with measured performance.

Day 9 — Sensor Fusion + Local Intelligence

• Combine PIR, IR sensor, and camera information.

• Create an event-confidence score.

• Example: PIR +30, IR +20, camera event +50.

• Classify events as LOW, MEDIUM, or HIGH confidence.

• Use confidence to decide whether to scan, capture, alert, or return to standby.

Deliverable: Sensor-fusion decision engine.

Day 10 — Edge Vision + Final Optimization

• Implement feasible local image processing such as resizing, grayscale conversion, frame differencing, or region-of-interest processing.

• Test whether TinyML/quantized inference is realistic for the exact ESP32 and available memory.

• Benchmark capture, image processing, event latency, RAM, flash, and CPU utilization.

• Record final measurements.

• Capture a clean demonstration of the finished system.

Deliverable: Finished Edge Vision Station with documented performance and a demo-ready build.

Final Resume / GitHub Checklist

■ Project runs reliably from a clean boot.

■ Only existing Sundoofer kit hardware is used.

■ Firmware is organized into clear modules/tasks.

■ FreeRTOS event-driven architecture is documented.

■ Camera capture and memory handling are reliable.

■ Sensor-fusion and decision logic are documented.

■ Latency and memory measurements are recorded.

■ GitHub README includes architecture, hardware, software, setup, and results.

■ 60–90 second engineering demo video is recorded.

■ Resume bullet uses actual measured results.

Suggested Technology Stack

C/C++ • ESP-IDF • FreeRTOS • GPIO • PWM • interrupts • timers • camera acquisition • sensor fusion • image processing • Git/GitHub